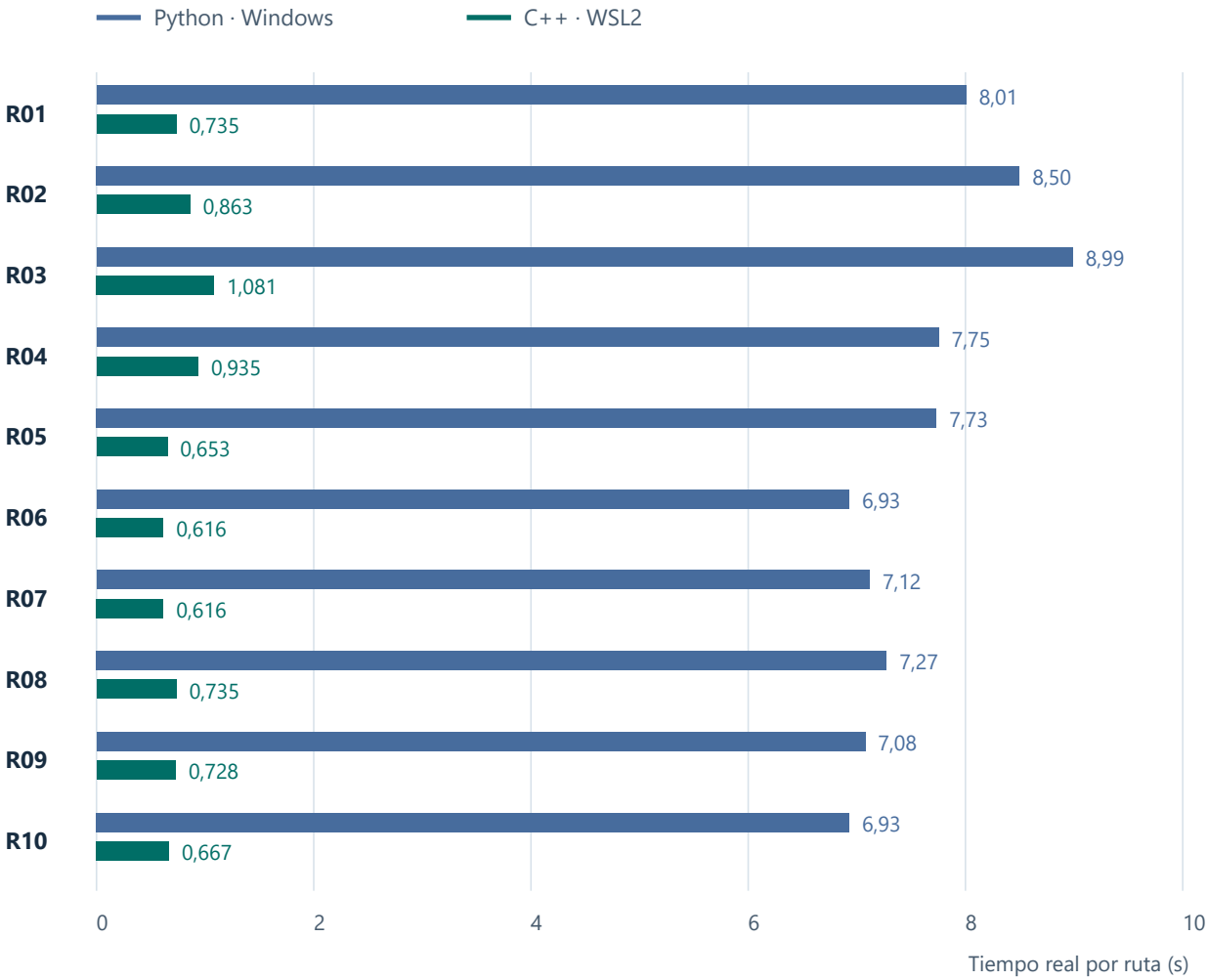
Rendimiento del motor

Diez rutas MAN10T sobre rimls.ply, de 10,00 s simulados cada una. Se emparejaron las entradas congeladas de la línea base de Python con la medición C++.

76,30 s
Python · suma del tiempo real

7,63 s
C++ · suma del tiempo real

10,0×
Cociente observado Python / C++



Motor	Mín. (s)	Mediana (s)	Media (s)	Máx. (s)	Completas
Python	6,928	7,502	7,630	8,986	10 / 10
C++	0,616	0,732	0,763	1,081	10 / 10

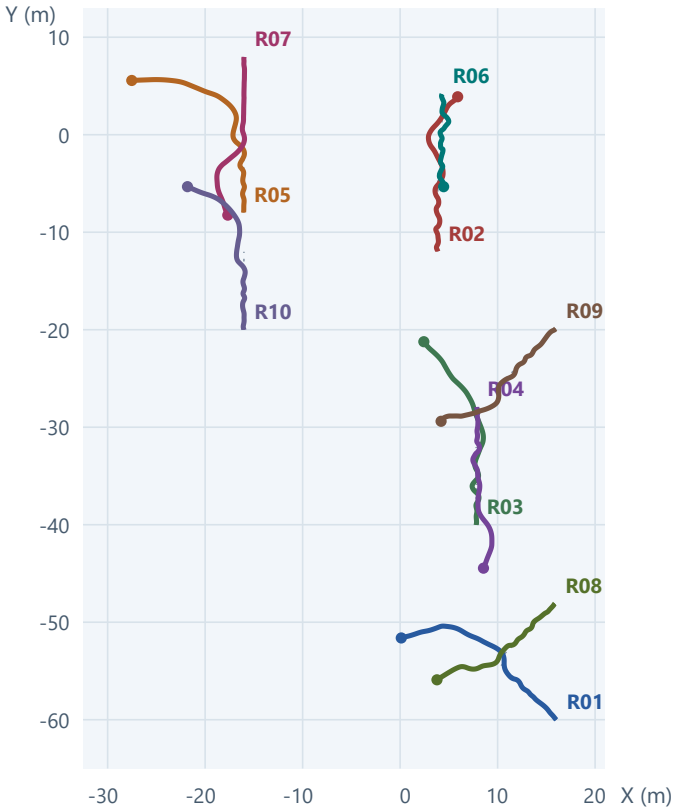
Comparación observacional

El cronómetro empieza con el mapa cargado. El resultado refleja dos entornos de ejecución, bibliotecas distintas y una medición por ruta. El cociente cuantifica el cambio de extremo a extremo de esta prueba.

Trayectorias y paridad observada

Los recorridos comparten el mapa y las poses de partida. Las líneas muestran el movimiento calculado y permiten comprobar cuánto se separan las dos soluciones.

Python — C++ — Guía de 8 m — Punto en el extremo C++



Ruta	Long. Py (m)	Long. C++ (m)	RMSE XY (mm)	Extremo (mm)	Máx. XY (mm)
R01	20,44	20,44	0,057	0,198	0,225
R02	17,93	17,93	0,028	0,075	0,075
R03	21,55	21,55	0,285	1,318	1,318
R04	17,54	17,54	0,011	0,035	0,040
R05	22,61	22,61	0,207	0,719	0,719
R06	10,61	10,61	0,011	0,035	0,035
R07	17,57	17,57	0,006	0,025	0,025
R08	15,40	15,40	0,022	0,068	0,069
R09	16,51	16,51	0,008	0,045	0,045
R10	18,12	18,12	0,088	0,041	0,393

Las líneas se superponen a escala del mapa. Máxima separación entre cuadros correspondientes: 1,318 mm. En 10/10 rutas coincidieron estado, avisos y número de muestras. Extremo mide la separación XY entre posiciones finales. El RMSE se calcula sobre los cuadros emparejados.

Uso de CPU y potencia de GPU

La dinámica se calculó en CPU. La potencia eléctrica disponible procede del sensor de la GPU NVIDIA del equipo anfitrión. La CPU queda expresada en tiempo y utilización.

Ruta	CPU Py (s)	CPU C++ (s)	CPU Py (%)	CPU C++ (%)	GPU Py (W)	GPU C++ (W)	Muestras GPU Py / C++
R01	15,453	0,706	8,8 %	4,4 %	3,43	11,51	40 / 4
R02	15,188	0,819	8,1 %	4,3 %	3,45	11,52	42 / 4
R03	17,812	1,031	9,0 %	4,3 %	3,46	11,55	44 / 5
R04	15,016	0,881	8,8 %	4,3 %	3,46	10,28	38 / 5
R05	16,984	0,615	10,0 %	4,3 %	3,44	3,66	38 / 3
R06	11,031	0,567	7,2 %	4,2 %	3,46	3,51	33 / 3
R07	14,766	0,568	9,4 %	4,2 %	3,45	3,48	35 / 3
R08	13,156	0,687	8,2 %	4,2 %	3,45	3,48	36 / 4
R09	12,781	0,685	8,2 %	4,3 %	3,43	3,47	35 / 4
R10	14,031	0,642	9,2 %	4,4 %	3,44	3,46	34 / 3

Lecturas agregadas

Motor	Rutas con W	Reposo (W)	Media (W)	Pico (W)	Energía (J)
Python	10 / 10	3,39	3,45	3,54	263,1
C++	10 / 10	35,10	6,59	11,55	54,9

Cómo leer los valores

CPU % = 100 × tiempo del proceso / (tiempo real × CPU lógicas). Denominador de 22 en Python y 22 en C++. GPU a intervalos solicitados de 200 ms: 375 lecturas en rutas Python y 38 en C++. La media resume las medias por ruta. La energía estimada integra las lecturas y contiene el consumo de fondo de la GPU.

La potencia eléctrica de CPU quedó sin medir. El reposo de GPU difirió entre sesiones y las ventanas de C++ reúnen de 3 a 5 muestras. Estas condiciones limitan la comparación del consumo de GPU entre motores.

Dónde se concentra el cálculo

Pasadas adicionales con instrumentación. Se ordenan funciones por tiempo propio dentro de cada perfil y se identifica su papel en el motor.

C++ · gprof

Función o símbolo	Propio (s)	Trabajo observado
CloudStore::neighbors	4,000	Busca vecinos de la nube y resuelve la superficie superior en XY.
TerrainPatch::search	0,720	Recorre el índice espacial del parche para hallar el punto más próximo.
std::__merge_sort_with_buffer	0,180	Ordena candidatos de la nube para agrupar consultas espaciales.
TerrainPatch::partition	0,170	Construye el índice espacial de los puntos del parche.
rigid::step	0,150	Calcula contactos, fuerzas e integración del cuerpo rígido.
TruckStepper::step	0,140	Combina terreno, geometría de contacto y avance de la dinámica.
TerrainPatch::covered_point	0,070	Comprueba la cobertura de la nube bajo cada contacto.

gprof muestreó cada 0,010 s y atribuyó 5,850 s. CloudStore::neighbors concentró el 68,4 % del tiempo propio registrado en 48 llamadas. Las funciones expandidas en línea pueden figurar bajo su llamante.

Python · cProfile

Función	Propio (s)	Acum. (s)	Trabajo observado
upper_xy	21,38	37,66	Selecciona los puntos de mayor cota para las consultas XY.
evaluate	7,05	92,63	Orquesta el bucle del motor, terreno y resultados.
q_to_R	5,88	6,29	Convierte cuaterniones en matrices de rotación.
<method 'reduce' of 'numpy.ufunc' objects>	4,22	4,22	Reduce arreglos NumPy mediante operaciones vectorizadas.
assemble_and_transform_robot	4,13	11,44	Transforma geometría, empujes e inercia.

Dos instrumentos de medida

Las pasadas perfiladas sumaron 9,436 s en C++ y 92,80 s en Python. gprof muestrea CPU nativa y cProfile instrumenta llamadas Python e incluye trabajo delegado a bibliotecas. Cada reparto de costes se interpreta dentro de su propio perfil.

Protocolo y trazabilidad

La línea base y la nueva ejecución conservan las entradas, los tiempos por ruta y los perfiles brutos para repetir la comparación.

Parámetro	Configuración
Escenario	rimls.ply · 366.797 puntos · MAN_10t · contactos completos
Control	R01 a R10 · poses asentadas · 0,01 s por paso · 10,00 s · CSV desactivado
Plataforma Python	Windows nativo · Python 3.12.10 · PyTorch 2.3.1+cu121 · 16 hilos
Plataforma C++	Linux-5.15.167.4-microsoft-standard-WSL2-x86_64-with-glibc2.39 · g++ (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0 · Release (-O3 -DNDEBUG)
Procesador	Intel(R) Core(TM) Ultra 7 155H · 22 lógicos en Python / 22 en C++
GPU	NVIDIA GeForce RTX 4050 Laptop GPU, 610.60
Plan de energía	Equilibrado

Qué incluye el cronómetro

Python mide la llamada al motor con la nube cargada. C++ mide el cálculo nativo y la construcción de resultados de reproducción en memoria. La carga del PLY, la serialización final, el navegador y el transporte quedan fuera.

Cómo repetir

Verificar la huella del PLY y mantener las diez entradas en orden. Cargar el mapa, calentar el motor durante 1 s simulado y medir 6 s de reposo de GPU. Hacer la pasada cronometrada con pausas de 2 s antes de cada ruta. Repetir con instrumentación según el README del paquete de medición.

Datos de referencia

Python: informe original y metrics.json de output/man10t-work. C++: metrics.json, gpu.csv y gprof.txt de output/man10t-profiling-cpp. Los campos de CPU describen el proceso cronometrado.

SHA-256 de rimls.ply

`054a48c406f01ca2fe3b464bd1ee30df9e9993828087035dc7235d63f7b67c91`